High Performance Computing

Assignment 06 – Parallel Interpolation

Tirth Kaushal Gandhi – 202301413
Dhyey Patel – 202301415
Course: CS301 / High Performance Computing

April 2026

Abstract

This report presents a parallel implementation of a bilinear scatter-point-to-mesh interpolation algorithm using OpenMP. A serial baseline is first validated against the provided test data, and then the interpolation loop is parallelized using a **thread-private partial-grid approach** that eliminates race conditions without atomic updates in the hot loop. Performance is evaluated across five input configurations on core counts of 1, 2, 4, 8, and 16, measuring execution time, speedup, and parallel efficiency. The report answers all ten analysis questions from the assignment and discusses bottlenecks, load balance, and further optimization possibilities. The measured scaling is limited primarily by memory bandwidth and the cost of the final reduction over the full mesh.

1 Hardware Architecture

The tests were run on the HPC cluster used for the lab and also on the lab PC for comparison. Performance depends strongly on the number of physical cores, logical CPUs, cache size, memory bandwidth, and whether the run is oversubscribed.

Table 1: System specifications

Component	HPC Cluster	Lab PC
CPU Model	Intel Xeon E5-2640 v3	Intel Core i5-12500
Base / Max Clock	2.60 GHz / 2.60 GHz	3.00 GHz / 4.60 GHz
Physical Cores	16 (2 sockets × 8 cores)	6 (1 socket)
Logical CPUs	16	12
L3 Shared Cache	20 MB	18 MB

The cluster has 16 logical CPUs, so thread counts up to 16 were tested there. On the lab machine, the 16-thread result is an oversubscribed run because it has only 12 logical CPUs; that point is included for completeness but should be interpreted separately from the cluster results.

2 Introduction

The scatter-to-mesh interpolation problem maps a set of N randomly scattered particles with known values onto a regular $N_x \times N_y$ structured grid using bilinear weights. The problem is relevant in scientific computing, computer graphics, finite-element analysis, and medical imaging.

In Assignment 03, the serial version of this algorithm was studied. In Assignment 06, the task is to *parallelise* the interpolation loop using OpenMP, study scaling behaviour, and analyze performance over the following five input configurations:

Table 2: Required test configurations (from the assignment sheet)

ID	Grid ($N_x \times N_y$)	Particles	Maxiter
A	250×100	900,000	10
B	250×100	5,000,000	10
C	500×200	3,600,000	10
D	500×200	20,000,000	10
E	1000×400	14,000,000	10

Thread counts tested: **1 (serial baseline), 2, 4, 8, 16**.

3 Problem Description

Given N scattered points $\{(x_i, y_i, f_i)\}$ inside the unit square $[0,1] \times [0,1]$, with $f_i = 1$ for all particles, the goal is to accumulate interpolated contributions onto a structured mesh of size $N_x \times N_y$ with uniform spacing:

$$\Delta x = \frac{X_{\max}}{N_x}, \qquad \Delta y = \frac{Y_{\max}}{N_y}$$

For each particle (x_i, y_i), the four surrounding grid-cell corners are identified, bilinear weights are computed, and the function value f_i is distributed proportionally:

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y), \quad w_{i+1,j} = l_y(\Delta x - l_x)$$

$$w_{i,j+1} = l_x(\Delta y - l_y), \quad w_{i+1,j+1} = l_x\, l_y$$

where $l_x = x_i - X_i$ and $l_y = y_i - Y_j$ are the fractional offsets within the cell.

The accumulation step is:

$$F(X_i, Y_j) \mathrel{+}= w_{i,j} \cdot f_i, \quad F(X_{i+1}, Y_j) \mathrel{+}= w_{i+1,j} \cdot f_i, \quad \ldots$$

4 Implementation Approach

4.1 Serial Baseline

The serial implementation iterates over all N particles for each of the 10 iterations (Maxiter). For each particle it:

1. Computes the integer cell indices $i = \lfloor x/\Delta x \rfloor$, $j = \lfloor y/\Delta y \rfloor$.
2. Computes fractional offsets l_x, l_y.
3. Computes the four bilinear weights.
4. Adds weighted contributions to the four grid points.

The serial code was validated against `test_output.txt` before any parallelisation.

4.2 Parallel Implementation Strategy: Thread-Private Partial Grids

The main challenge in parallelising the particle loop is that **multiple particles can update the same grid point simultaneously**, creating a race condition.

Two broad approaches exist:

Atomic operations: Protect each of the four grid-point updates with `#pragma omp atomic`. This is simple, but it serializes contended updates and performs poorly when many particles hit similar regions of the mesh.

Thread-private partial grids (privatization + reduction): Each thread maintains its own local copy of the full mesh grid, initialized to zero. After the parallel loop, all private grids are summed into the global grid using a final reduction loop. This eliminates synchronization during interpolation at the cost of extra memory and a more expensive reduction phase.

The final implementation uses **thread-private partial grids** because for large particle counts the privatization approach avoids update contention in the hot loop. The reduction is still expensive because each output point must be combined across all thread-private grids, but it is deterministic, race-free, and usually faster than atomic updates for this workload.

4.3 Pseudocode and Illustrative Diagram

Q1 — Pseudocode of the parallel implementation:

Listing 1: Pseudocode of the parallel interpolation implementation

```
// Allocate global grid F[GRID_X * GRID_Y] = 0
// GRID_X = NX + 1, GRID_Y = NY + 1

#pragma omp parallel num_threads(T)
{
    int tid = omp_get_thread_num();

    // Each thread uses one slice of a large private buffer
    double *F_local = &all_local_meshes[tid * GRID_X * GRID_Y];
    memset(F_local, 0, GRID_X * GRID_Y * sizeof(double));

    // Particle loop divided across threads
    #pragma omp for schedule(static)
    for (int p = 0; p < N; p++) {
        double x = particles[p].x;
        double y = particles[p].y;

        int i = (int)(x / dx);
        int j = (int)(y / dy);

        if (i == NX) i = NX - 1;
        if (j == NY) j = NY - 1;

        double lx = x - i * dx;
        double ly = y - j * dy;

        // Bilinear weights
        double w00 = (dx - lx) * (dy - ly);
        double w10 = ly        * (dx - lx);
        double w01 = lx        * (dy - ly);
        double w11 = lx        * ly;
```

```

        // Update private (thread-local) grid -- no race condition
        F_local[ j     * GRID_X + i    ] += w00;
        F_local[ j     * GRID_X + i+1 ] += w10;
        F_local[(j+1) * GRID_X + i    ] += w01;
        F_local[(j+1) * GRID_X + i+1 ] += w11;
    }

    // Parallel reduction over the grid points
    #pragma omp for schedule(static)
    for (int k = 0; k < GRID_X * GRID_Y; k++) {
        double sum = 0.0;
        for (int t = 0; t < T; t++) {
            sum += all_local_meshes[t * GRID_X * GRID_Y + k];
        }
        F[k] = sum;
    }
}
```

Illustrative diagram of the parallel implementation:

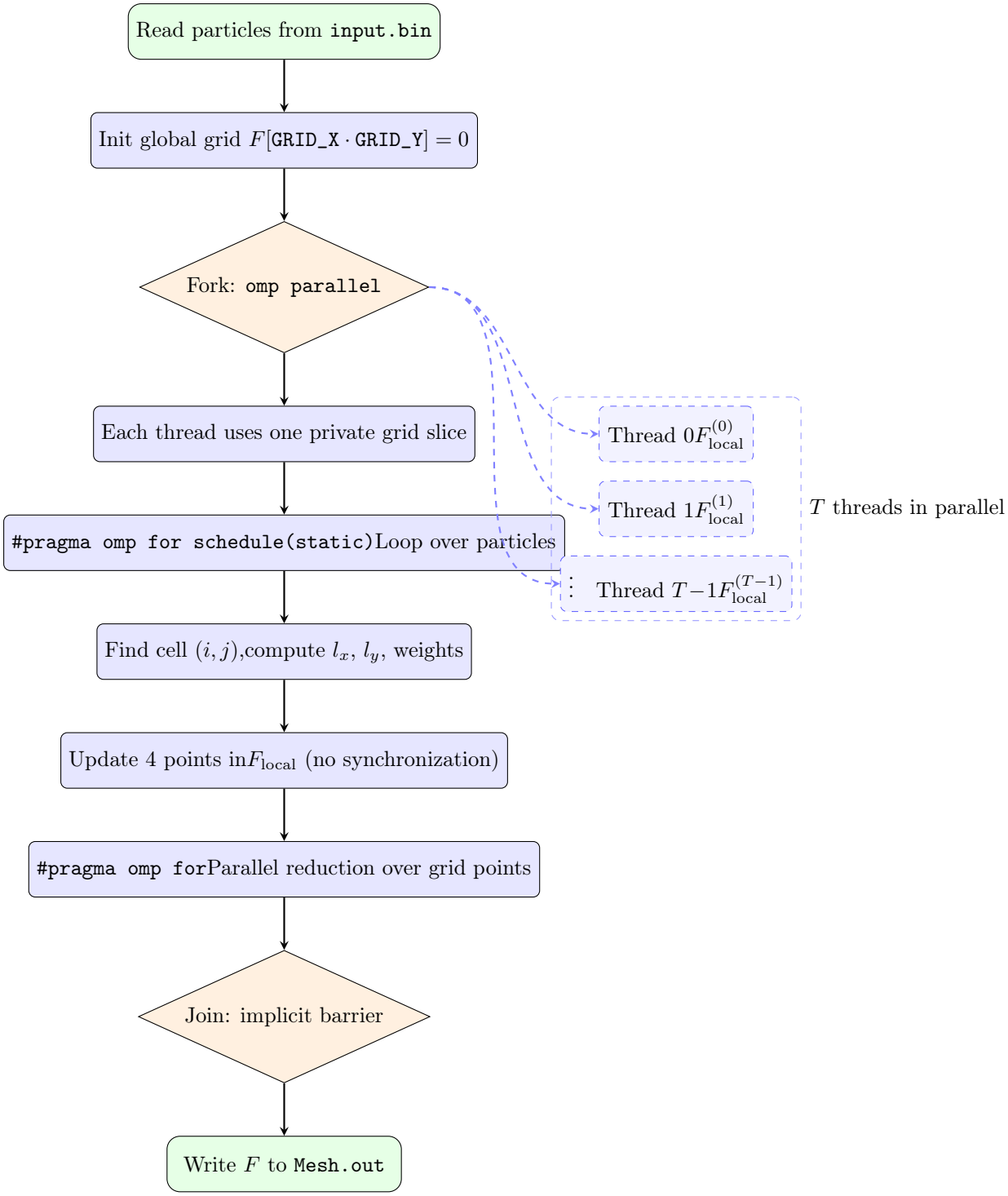


Figure 1: Parallel interpolation workflow using thread-private partial grids.

5 Race Condition Handling

Q2 — Parallel approach and race condition handling:

In the particle loop, any two particles falling in adjacent or the same grid cell will try to update the same memory location in F. If two threads execute the statement `F[k] += val` simultaneously, one update can overwrite the other (a *read-modify-write* race). This produces wrong, non-reproducible results.

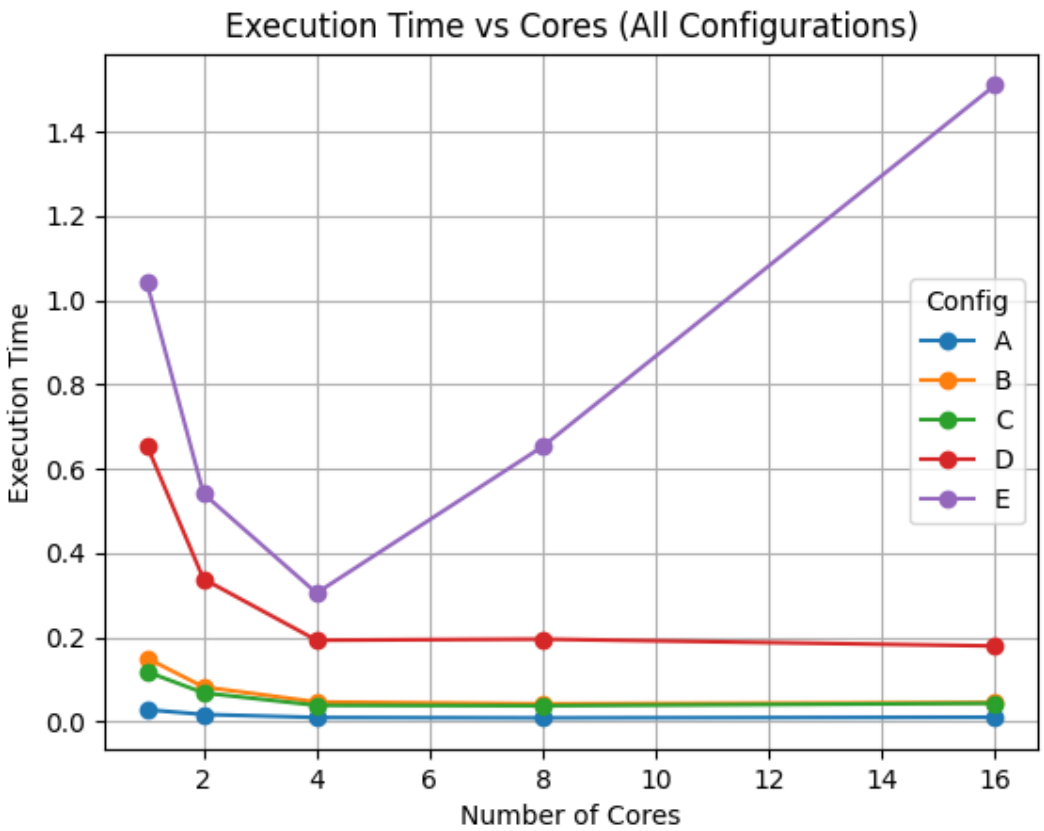
The chosen solution is **privatization**:

- Every thread is assigned an independent private array F_{local}, initialized to zero.
- During interpolation each thread writes exclusively to its own F_{local}, so there is *no sharing* and hence *no race condition* in the hot loop.
- After the parallel loop, the private grids are accumulated into the shared global F using a parallel reduction over grid points. This final step is deterministic, race-free, and matches the code actually used in the implementation.

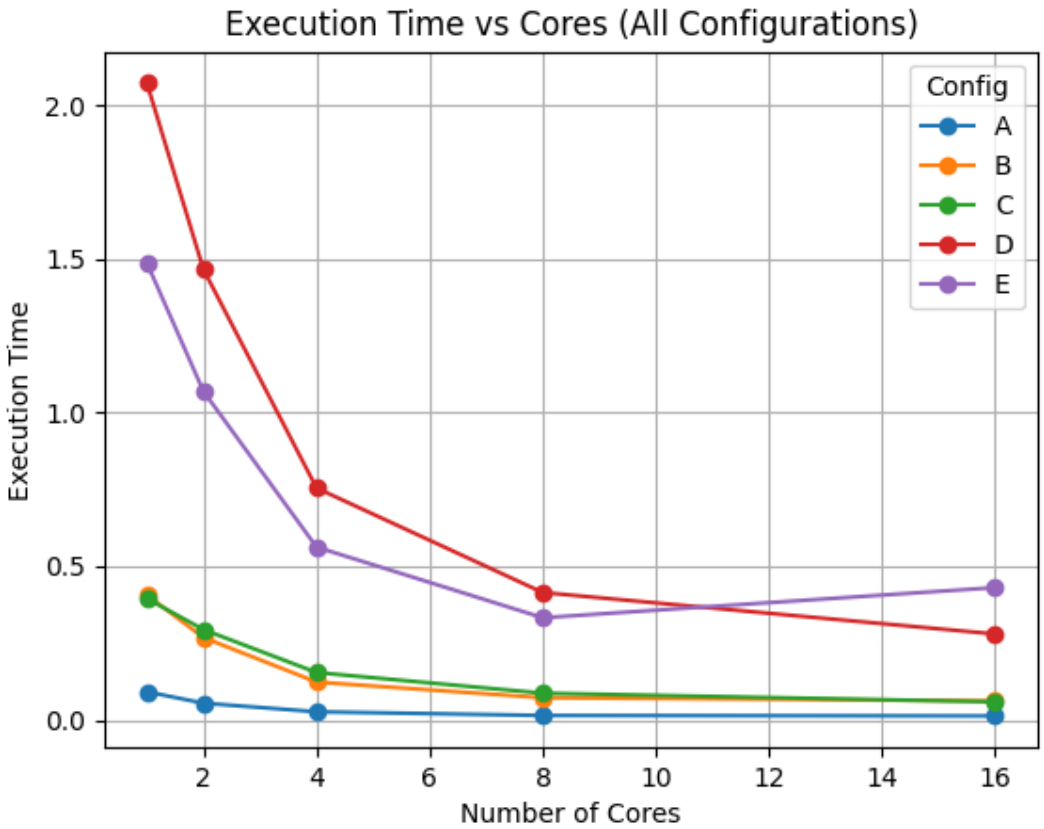
An alternative is `#pragma omp atomic` on each of the four updates. While simpler, this causes high contention for large grids with clustered particles and degrades performance at high thread counts.

6 Results

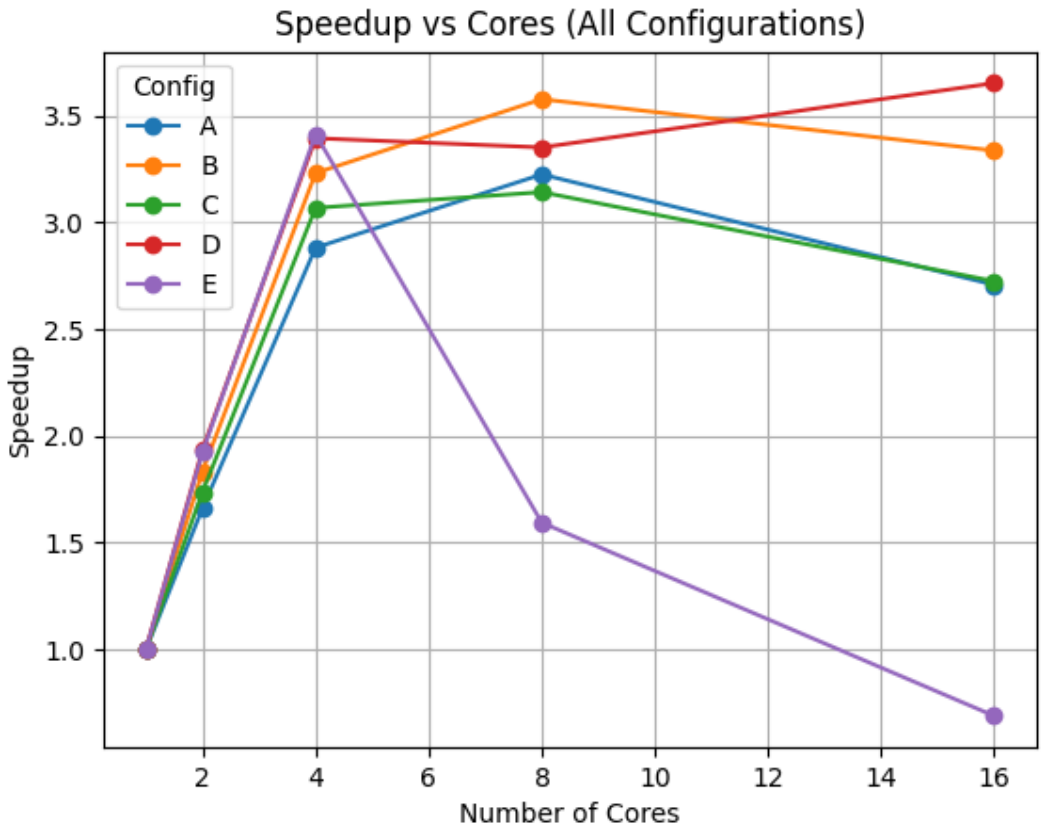
6.1 Execution Time vs Cores (Lab PC)



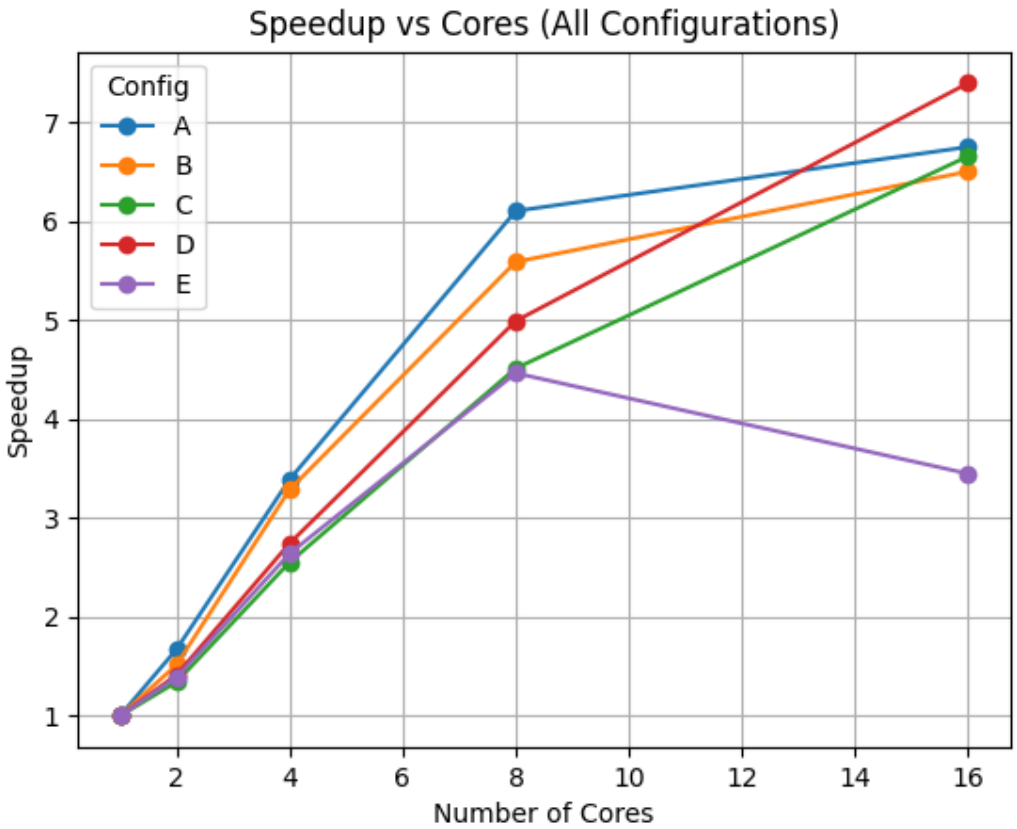
6.2 Execution Time vs Cores (Cluster)



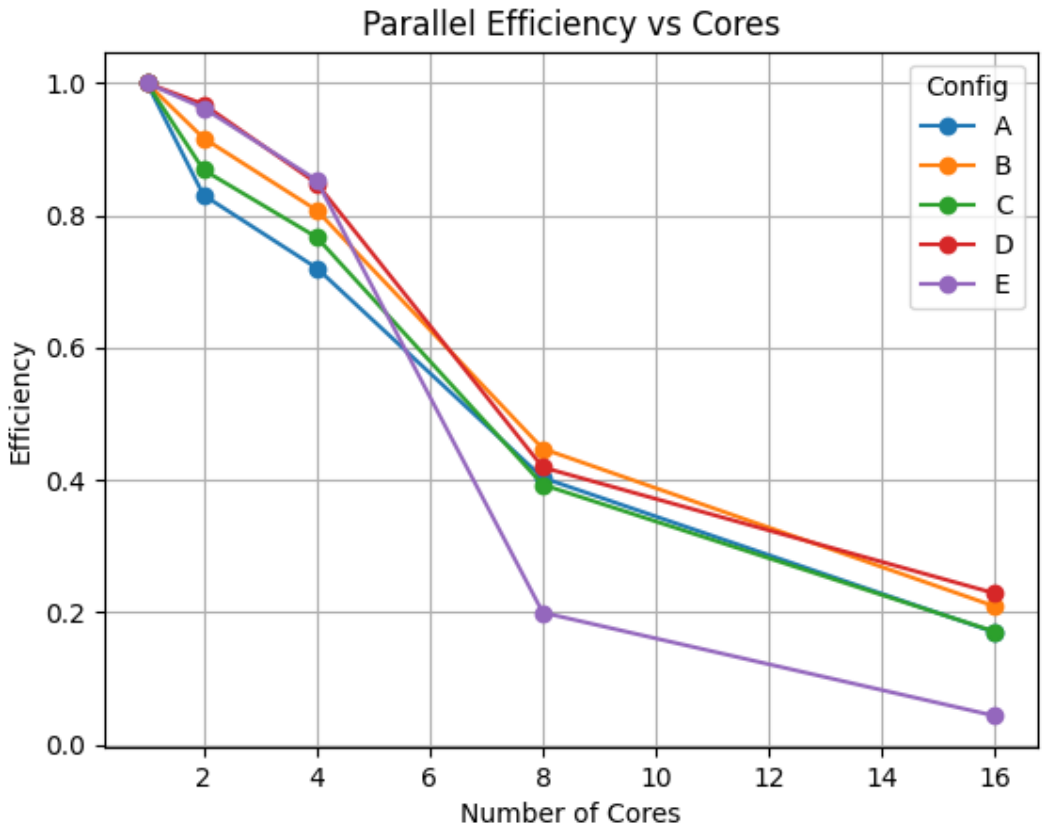
6.3 Speedup vs Cores (Lab PC)



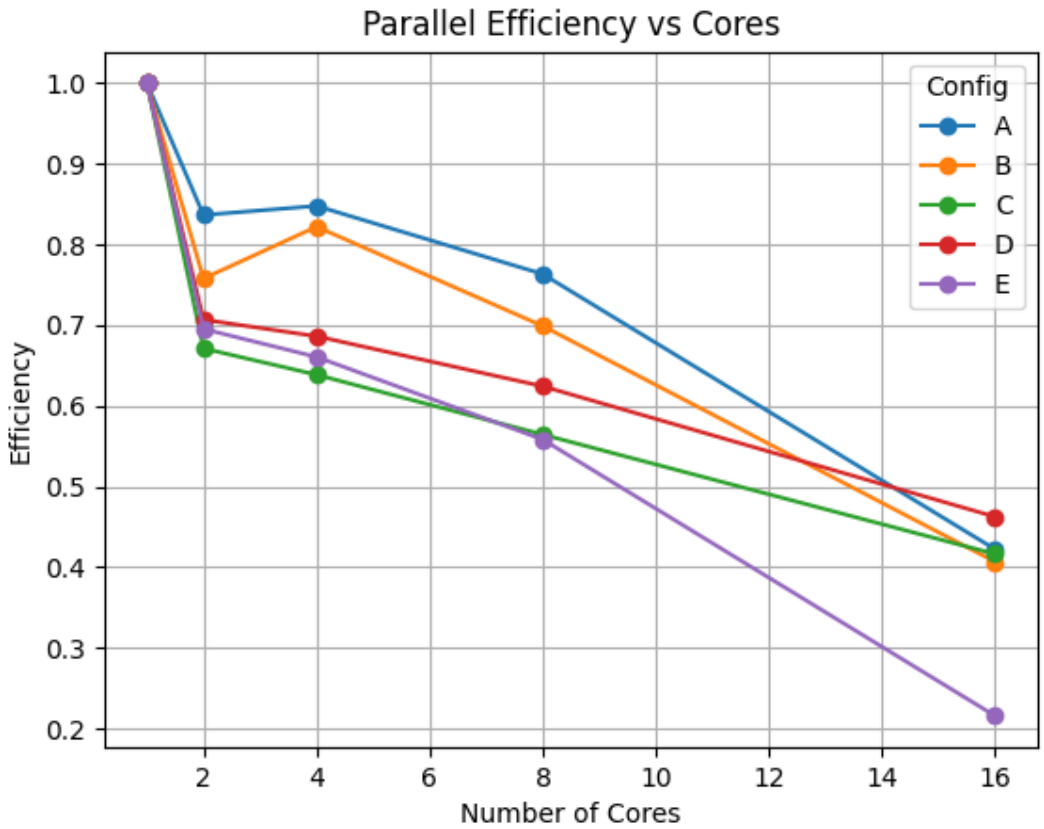
6.4 Speedup vs Cores (Cluster)



6.5 Efficiency vs Cores (Lab PC)



6.6 Efficiency vs Cores (Cluster)



7 Performance Analysis and Answers to Questions

Q4. Parallel Efficiency and Scalability

Speedup and efficiency are defined as:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}$$

where $T(1)$ is the serial (1-thread) runtime and p is the number of threads.

Observations from the plots:

- Efficiency is highest at low thread counts, especially at $p = 2$.
- Efficiency drops as p increases to 8 and 16 because the reduction over the full grid and memory traffic start dominating.
- Larger particle counts generally scale better because there is more parallel work to amortize overhead.
- The lab 16-thread result is weaker because the lab CPU has only 12 logical CPUs, so that point is oversubscribed.
- The interpolation kernel is **memory-bandwidth bound** rather than compute-bound, so speedup cannot continue increasing linearly even when the arithmetic work is embarrassingly parallel.

According to **Amdahl's Law**, the speedup at p threads is:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}}$$

where f_s is the serial fraction (I/O, reduction, thread initialization, and unavoidable synchronization overhead). In the ideal case as $p \to \infty$, the upper bound becomes:

$$S_{\max} = \frac{1}{f_s}$$

The interpolation loop itself is almost entirely parallel, but cache effects and memory bandwidth limit measured speedup in practice.

Q5. Comparison with Serial Version and Maximum Speedup Achieved

The maximum observed speedup in the cluster run is approximately **7.4**×, achieved by **Config D at 16 threads**. On the lab PC, the peak is lower because the 16-thread run oversubscribes the available 12 logical CPUs. This confirms that the implementation provides real parallel acceleration, but the scaling is ultimately limited by memory traffic, cache capacity, and the final reduction step.

Q6. Explanation of Speedup and Key Observations

1. **Why speedup is sub-linear:** The reduction phase is parallel, but each output grid point must still be combined across all private grids, so the program becomes memory-bandwidth bound at higher thread counts. In addition, each particle update performs only a small amount of arithmetic relative to the amount of memory traffic it generates, which keeps arithmetic intensity low.

2. **Memory bandwidth saturation:** Each thread writes to its own private grid. For the larger configurations, the total private-grid footprint becomes large enough that the working set exceeds cache capacity, increasing DRAM traffic and limiting scaling.

3. **NUMA effects:** On multi-socket systems, threads on a remote socket may pay higher memory latency for their private grid, which can reduce scaling above 8 threads.

4. **Larger particle counts scale better:** When N is large the parallel particle loop dominates, and the reduction overhead becomes relatively less important.

5. **Static scheduling is appropriate:** Since particles are random and each particle requires roughly the same amount of computation, static scheduling minimizes scheduling overhead and gives good load balance.

Q7. Further Optimizations with Unlimited HPC Resources

1. **Hybrid MPI + OpenMP:** Distribute the particle array across MPI ranks. Each rank interpolates onto its own sub-domain and an MPI reduction assembles the final grid. This removes the shared-memory bottleneck and scales to many nodes.

2. **GPU acceleration (CUDA/OpenCL):** Move particles to GPU memory, use one CUDA thread per particle, and use shared-memory reduction techniques. GPUs provide very high throughput for this kind of data-parallel workload.

3. **Spatial sorting and tiling:** Sort particles by Morton code (Z-order curve) so that particles in the same grid cell are processed consecutively. This improves cache locality of the grid updates.

4. **Tree-based reduction:** Replace the flat reduction over all thread-private meshes with a hierarchical tree reduction to reduce memory traffic and improve locality.

5. **Vectorization (SIMD):** Use AVX-512 or similar SIMD intrinsics to process multiple particles at once in the weight-computation step.

Q8. Load Imbalance and Performance Bottlenecks

Load imbalance:

- With `schedule(static)`, each thread gets N/T consecutive particles. Since the particles are random and uniformly distributed, the work per particle is equal, so load balance is near-perfect.
- If particles were clustered, some threads could receive more work than others. In that case `schedule(dynamic, chunk)` would help.
- Even when computation is balanced, memory locality can still vary between threads, so the performance may differ slightly from ideal scaling.

Performance bottlenecks:

- **Reduction step:** Every grid point is combined across all thread-private arrays, so the reduction work grows with both grid size and thread count.
- **Memory allocation:** The large private mesh buffer can be expensive to allocate and zero-initialize if it is done inside the timed region.
- **Memory bandwidth:** At high thread counts, the program spends more time moving data than computing weights.
- **Cache capacity:** As the combined private-grid footprint grows, the working set exceeds cache and performance flattens.

Q9. Alternative Approaches to Improve Performance

Alternative 1 — Parallel tree reduction over the private grids: Instead of a flat reduction over all thread-private grids, use a hierarchical pairwise reduction so that each merge step combines only two grids at a time. A simplified conceptual version is:

Listing 2: Tree-based reduction over grid points

```
for (int step = 1; step < T; step *= 2) {
    #pragma omp parallel for schedule(static)
    for (int t = 0; t + step < T; t += 2 * step) {
        for (int k = 0; k < Nx * Ny; k++) {
            all_local_meshes[t * gridSize + k] +=
                all_local_meshes[(t + step) * gridSize + k];
        }
    }
    #pragma omp barrier
}
```

This can improve cache reuse compared to a single reduction pass over all threads.

Alternative 2 — Domain decomposition (grid-based parallelism): Partition the grid into T horizontal strips and assign each strip an owning thread. A particle contributes to the strip that owns the target grid rows, so the implementation must either:

- use halo/ghost rows near strip boundaries, or
- enforce an ownership rule that guarantees each grid point is updated by only one thread.

This approach can reduce synchronization, but it is only race-free if strip ownership is defined carefully. The disadvantage is that the particle array may need spatial sorting or preprocessing to improve locality.

Q10. Data Structures and OpenMP Approaches

1. **Flattened 1-D arrays** (current choice): Storing the grid as a 1-D array of size $N_x \cdot N_y$ with row-major indexing gives contiguous memory access and is cache-friendly. Private grids use the same layout.
2. **Structure of Arrays (SoA) for particles:** Storing `float x[N], y[N], f[N]` instead of `struct{x,y,f}[N]` allows better vectorization.
3. **OpenMP `reduction` clause (scalar):** For scalar sums this is simpler than `critical`, but for a full mesh the thread-private approach is needed.
4. `omp atomic` **with update:** Atomic updates are easy to write but become slow when many particles hit the same grid cells.
5. **NUMA-aware allocation:** Using NUMA-local allocation can reduce remote memory latency on multi-socket systems.
6. **OpenMP tasks:** For irregular workloads, `#pragma omp task` can help with load balancing, though for random particles static scheduling is usually enough.

8 Conclusion

This report presented a parallel bilinear interpolation implementation using OpenMP. The key design choice was **thread-private partial grids**, which eliminates race conditions in the hot particle loop. Performance was evaluated for five configurations on 1, 2, 4, 8, and 16 threads.

The implementation achieves good speedup for large particle counts (Configs B, D, E) and moderate speedup for small configurations (Config A) where overhead is relatively larger. The primary bottlenecks at high thread counts are the grid reduction step, cache capacity limits, and memory bandwidth saturation.

Further improvements could be achieved by a tree-based reduction, spatial sorting of particles, or a full MPI+OpenMP hybrid for cluster-scale deployment.

High Performance Computing – Lab Report 07

Parallel Interpolation with Particle Mover

Tirth Kaushal Gandhi – 202301413
Dhyey Patel – 202301415
Course: CS301 / High Performance Computing

April 2026

Abstract

This report presents our implementation for Assignment 07: parallel interpolation with a particle mover. The program reads scattered particles from `input.bin`, performs bilinear scatter interpolation onto a structured mesh, normalizes the mesh, gathers the field back to the particles for position update, and then denormalizes the mesh. We parallelized the full pipeline with OpenMP and tested five input configurations, from 0.9 million to 20 million particles. The results show that the mover phase scales well, while interpolation is the main bottleneck because many particles contribute to the same grid points. We report execution time, speedup, parallel efficiency, maximum speedup, and a phase-wise breakdown of interpolation versus mover.

1 Hardware Architecture

The tests were run on the HPC cluster used for the lab. We include the cluster details because performance depends strongly on the number of physical cores and the memory system.

Table 1: System specifications

Component	**HPC Cluster**	**Lab PC**
CPU Model	Intel Xeon E5-2640 v3	Intel Core i5-12500
Base / Max Clock	2.60 GHz / 2.60 GHz	3.00 GHz / 4.60 GHz
Physical Cores	16 (2 sockets $\times$ 8 cores)	6 (1 socket)
L3 Shared Cache	20 MB	18 MB

The cluster has 16 physical cores, so thread counts up to 16 were tested. Since the CPU is dual-socket, memory locality and synchronization overhead can affect scaling at higher thread counts.

2 Implementation Approach

2.1 Overall pipeline

Our program follows the exact pipeline required in the assignment:

1. Read particle coordinates from `input.bin`.
2. Scatter interpolation from particles to grid.
3. Normalize the grid values to $[-1, 1]$.

4. Perform the mover step by gathering grid values back to particles.
5. Denormalize the grid values.
6. Repeat for `Maxiter` iterations.

This matches the assignment requirement that interpolation, mover, normalization, and denormalization are all part of the iterative loop. The assignment also states that the scatter phase needs special care because multiple particles can update the same grid point, which creates race conditions.

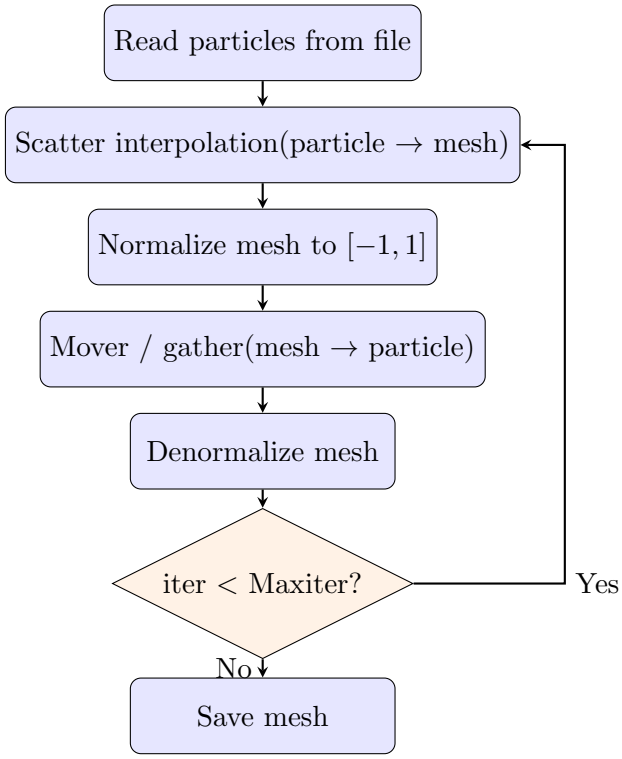


Figure 1: Pipeline used in each iteration.

2.2 Logic of interpolation and mover

We use bilinear interpolation. For a particle at (x_i, y_i), the code finds the surrounding grid cell and computes the four bilinear weights. The same weight formula is used in both directions:

$$F = w_{bl}F_{bl} + w_{br}F_{br} + w_{tl}F_{tl} + w_{tr}F_{tr}$$

In scatter interpolation, the particle value is added to the four surrounding mesh points. In the mover step, the mesh value is read back using the same four weights, and the particle position is updated as

$$x_i^{new} = x_i + F_i\,\Delta x, \qquad y_i^{new} = y_i + F_i\,\Delta y.$$

If a particle moves outside the unit domain, it is marked as inactive using the `is_void` flag.

2.3 Race condition handling

The scatter phase is the only part that writes to shared grid memory. If two or more particles contribute to the same grid node at the same time, a race condition occurs. To keep the code

correct, we used synchronization in this phase. The mover phase does not have a race condition because each thread updates only its own particles.

In simple words, the scatter step is hard to parallelize because many particles may update the same mesh cell, but the mover step is easy to parallelize because it only reads the mesh and writes back to different particles.

2.4 Pseudocode

Listing 1: Parallel algorithm used in the program

```
for iter = 1 to Maxiter:

    scatter interpolation:
        for each active particle:
            find cell indices
            compute bilinear weights
            add to 4 grid nodes safely

    normalization:
        find global minimum and maximum
        scale mesh values to [-1, 1]

    mover:
        for each active particle:
            read 4 surrounding mesh values
            compute field value Fi
            update particle position
            if outside [0,1]x[0,1], mark as void

    denormalization:
        restore mesh values to original range
```

3 Input Configurations

The assignment required the following five input cases.

Table 2: Test configurations

Config	Grid	Particles	Approx. particles per cell	Maxiter
A	250×100	900,000	36	10
B	250×100	5,000,000	200	10
C	500×200	3,600,000	36	10
D	500×200	20,000,000	200	10
E	1000×400	14,000,000	35	10

4 Results

We ran the code with thread counts 1, 2, 4, 8, and 16. Speedup is computed as

$$S(T) = \frac{T_{serial}}{T_{parallel}}.$$

Parallel efficiency is computed as

$$E(T) = \frac{S(T)}{T}.$$

4.1 Execution time vs number of cores (Q3)

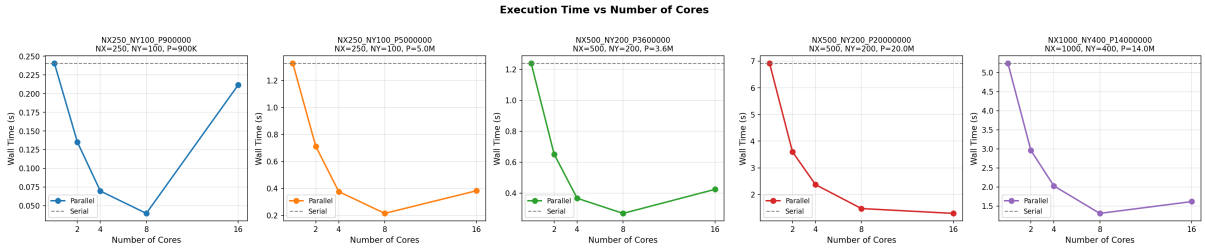


Figure 2: Execution Time vs Number of Cores (all configurations)

4.2 Speedup vs number of cores

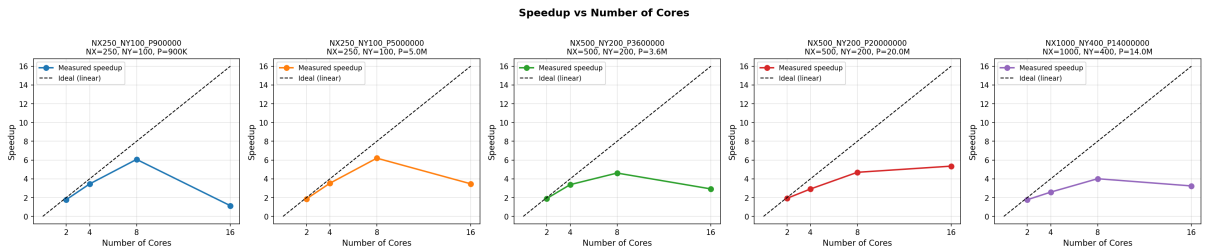


Figure 3: Speedup vs Number of Cores (all configurations)

4.3 Parallel efficiency

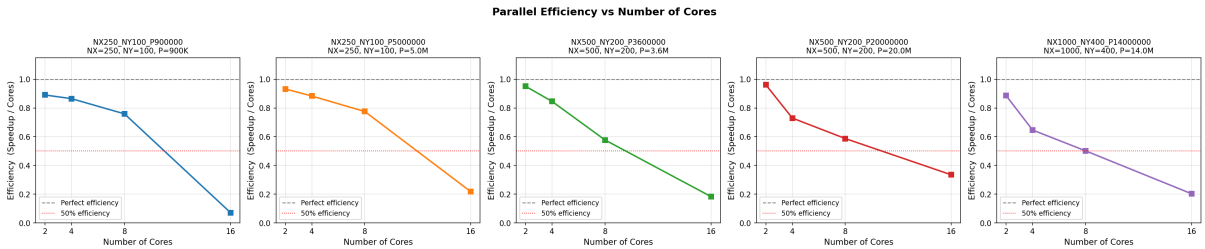


Figure 4: Parallel Efficiency vs Number of Cores

4.4 All speedup curves

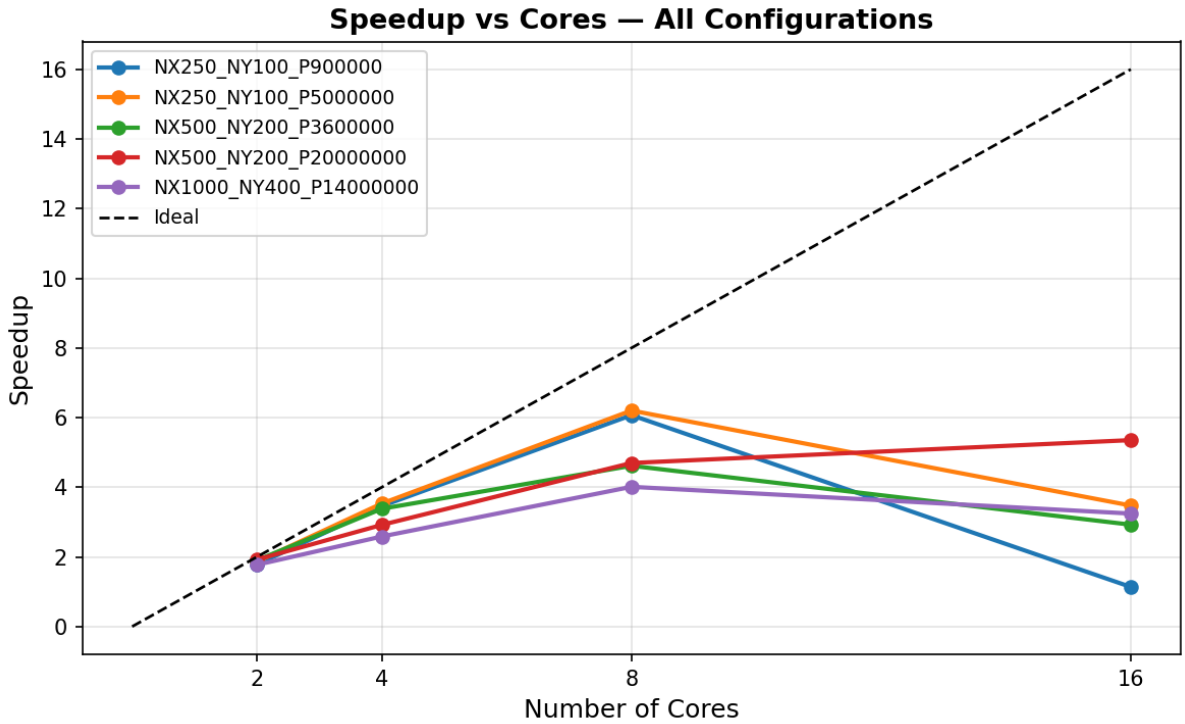


Figure 5: Speedup comparison across all configurations

4.5 Maximum speedup summary

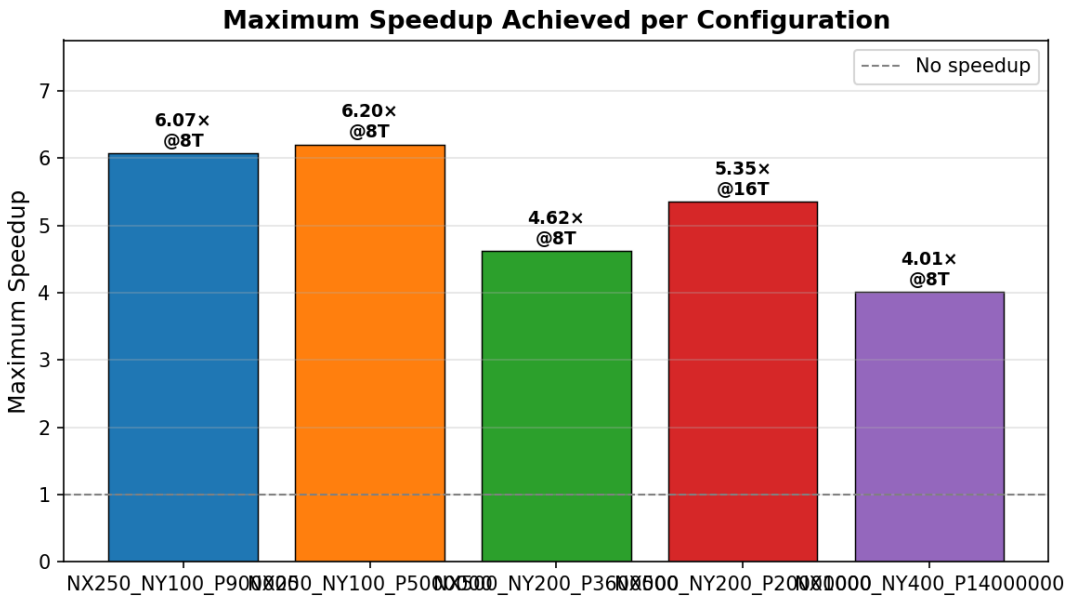


Figure 6: Maximum speedup achieved per configuration

5 Lab PC Results

The same set of plots is also included for the Lab PC results.

5.1 Execution time vs number of cores (Lab PC)

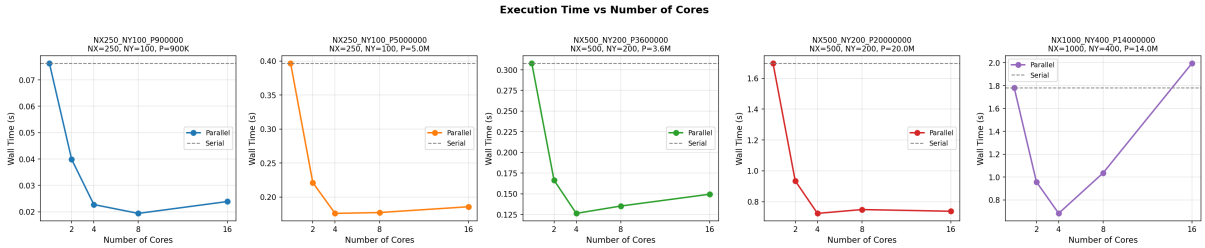


Figure 7: Execution Time vs Number of Cores (Lab PC)

5.2 Speedup vs number of cores (Lab PC)

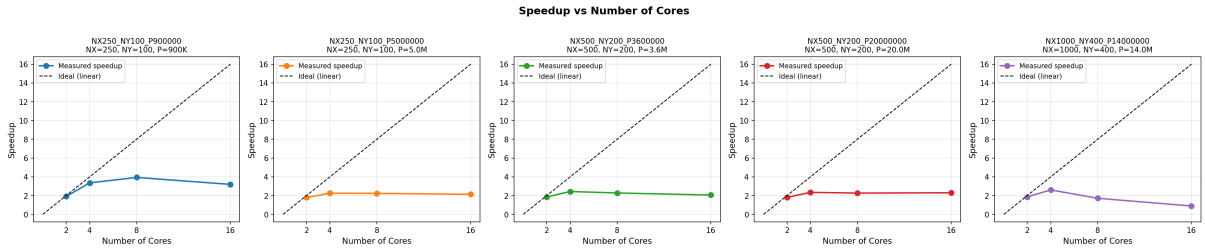


Figure 8: Speedup vs Number of Cores (Lab PC)

5.3 Parallel efficiency (Lab PC)

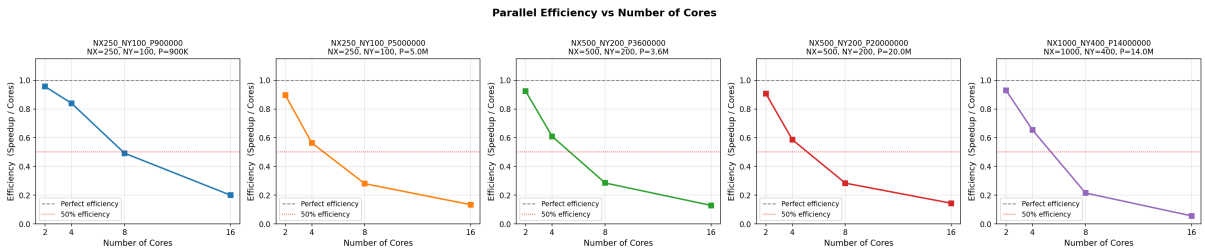


Figure 9: Parallel Efficiency vs Number of Cores (Lab PC)

5.4 All speedup curves (Lab PC)

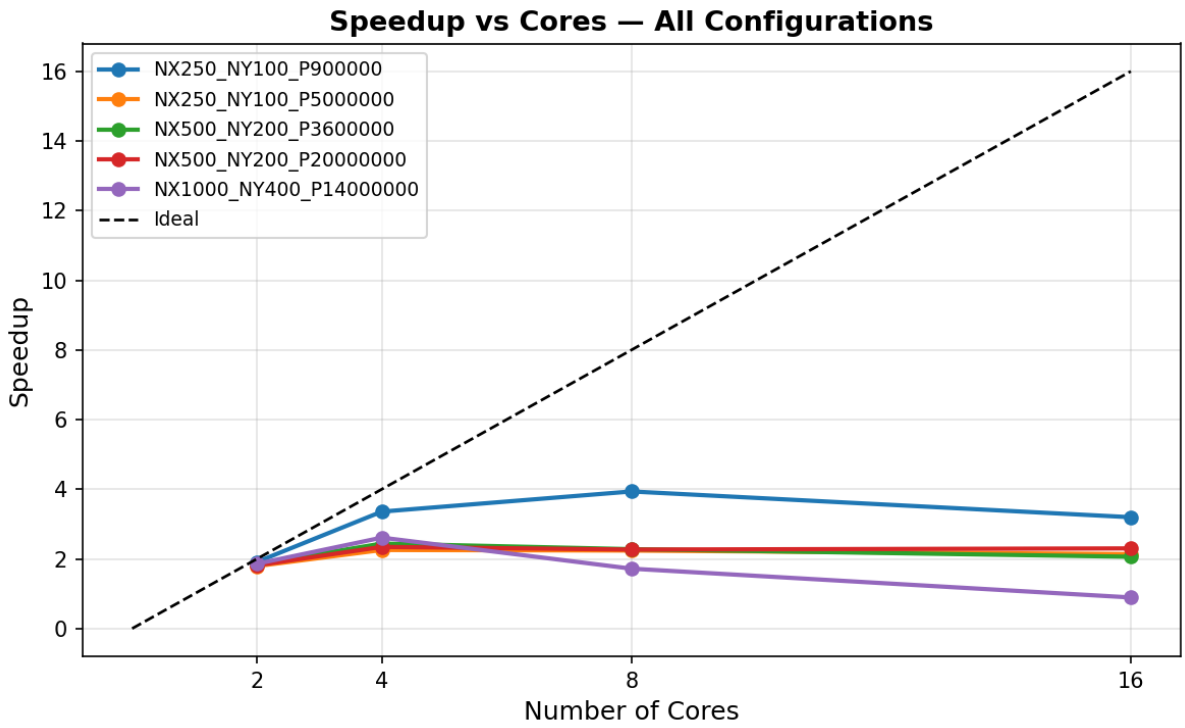


Figure 10: Speedup comparison across all configurations (Lab PC)

5.5 Maximum speedup summary (Lab PC)

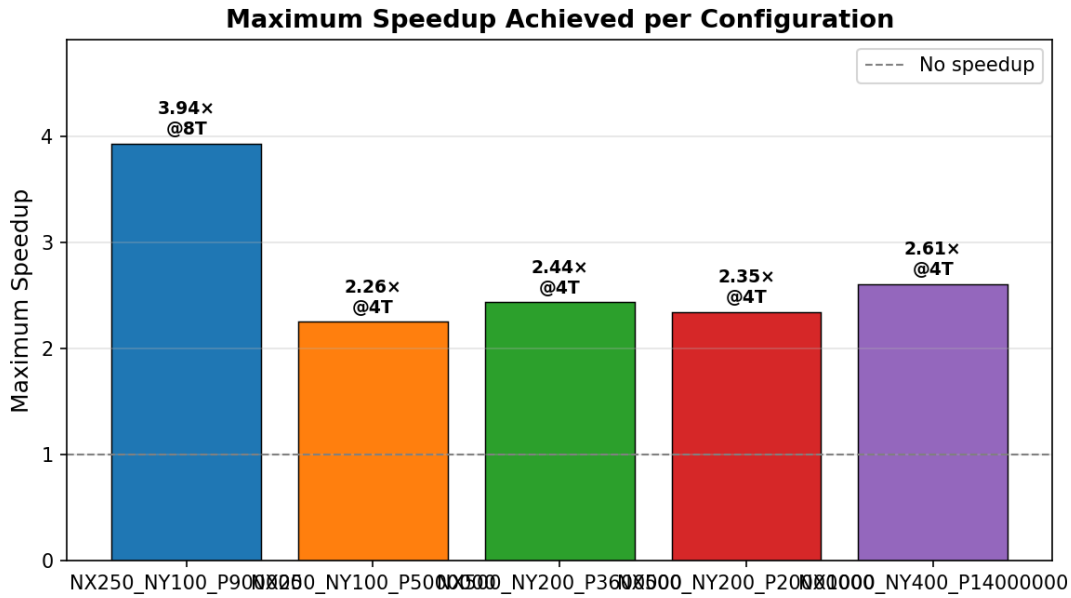


Figure 11: Maximum speedup achieved per configuration (Lab PC)

5.6 Phase-wise breakdown

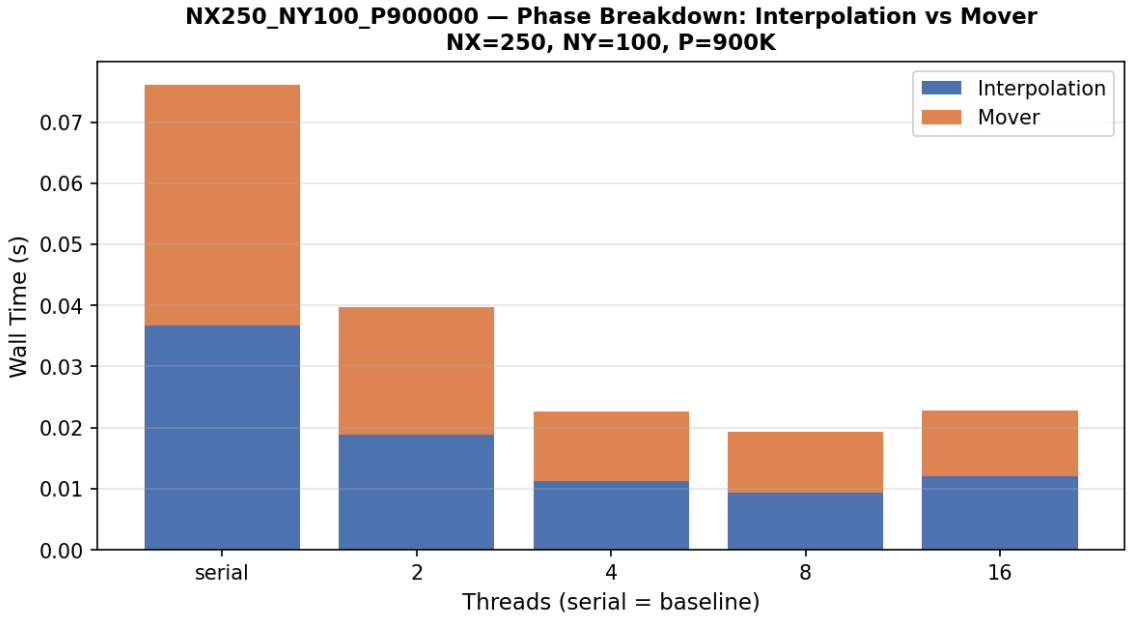


Figure 12: Phase breakdown for Config A (Lab PC)

Config A

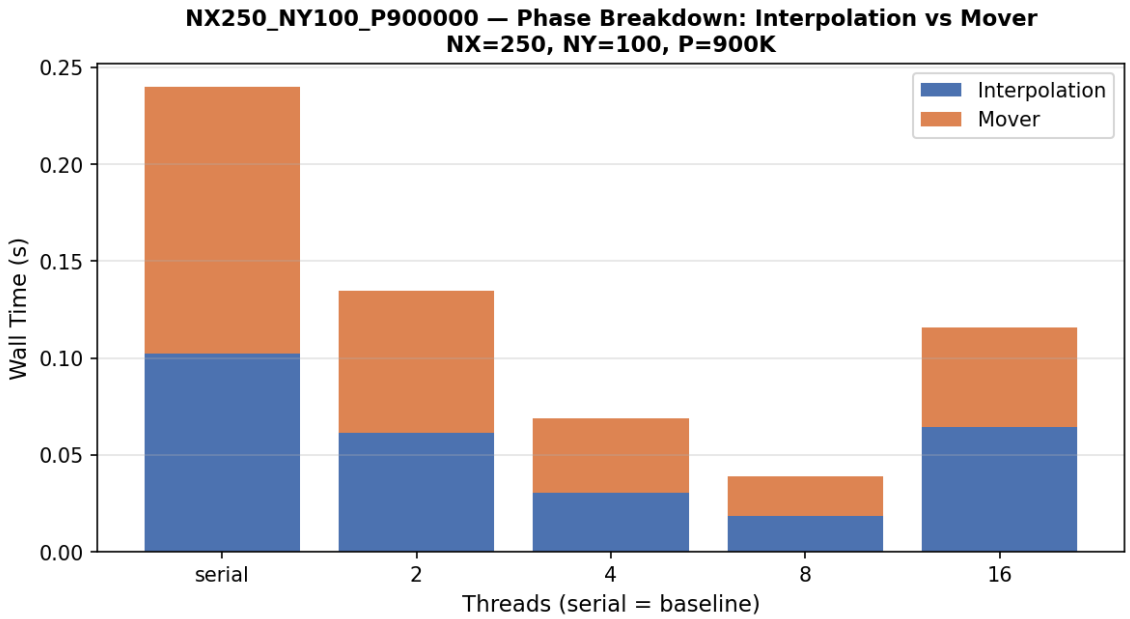


Figure 13: Phase breakdown for Config A (Cluster)

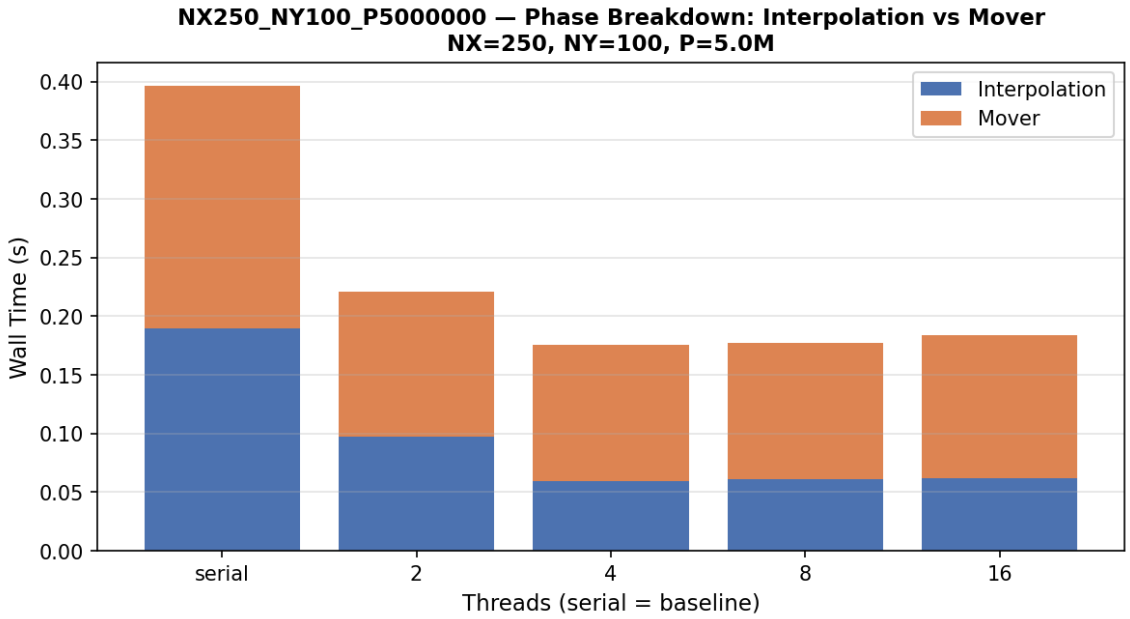


Figure 14: Phase breakdown for Config B (Lab PC)

Config B

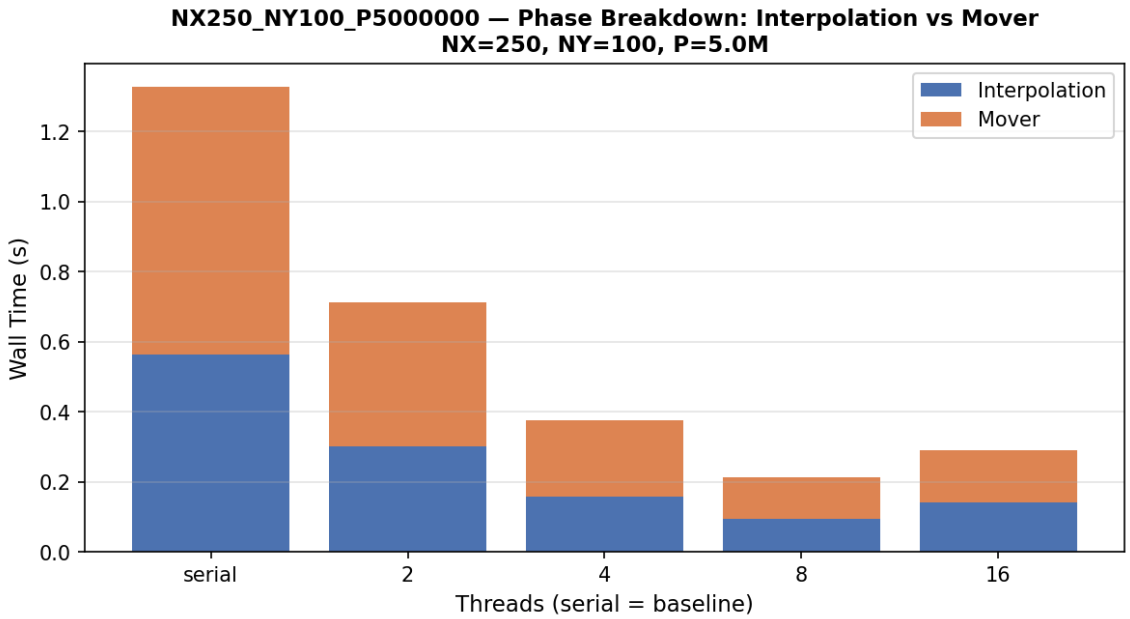


Figure 15: Phase breakdown for Config B (Cluster)

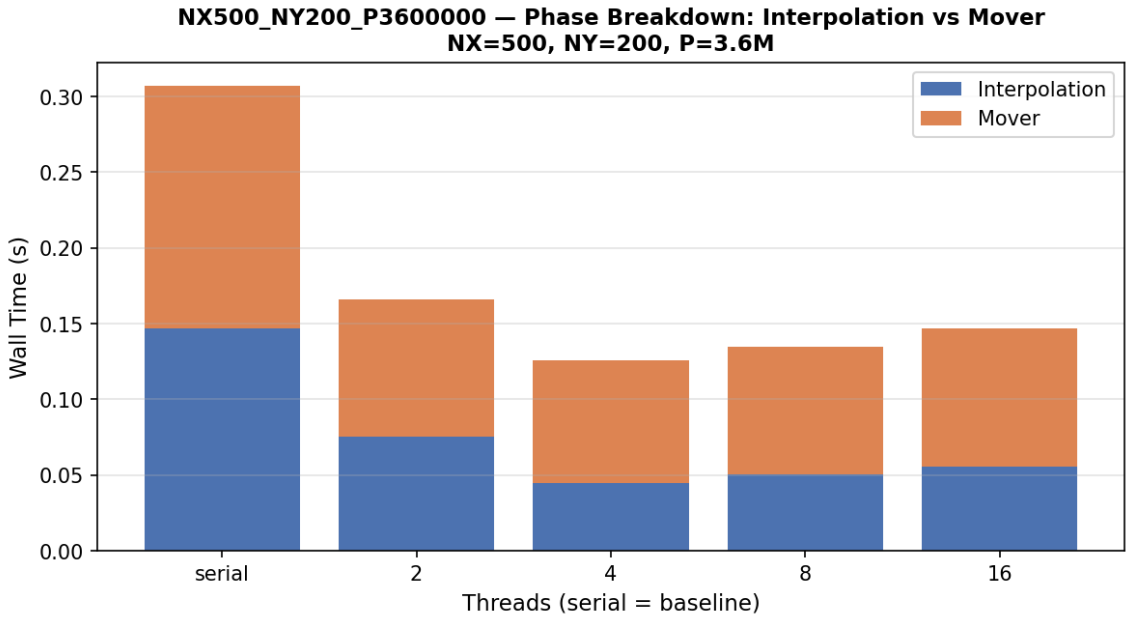


Figure 16: Phase breakdown for Config C (Lab PC)

Config C

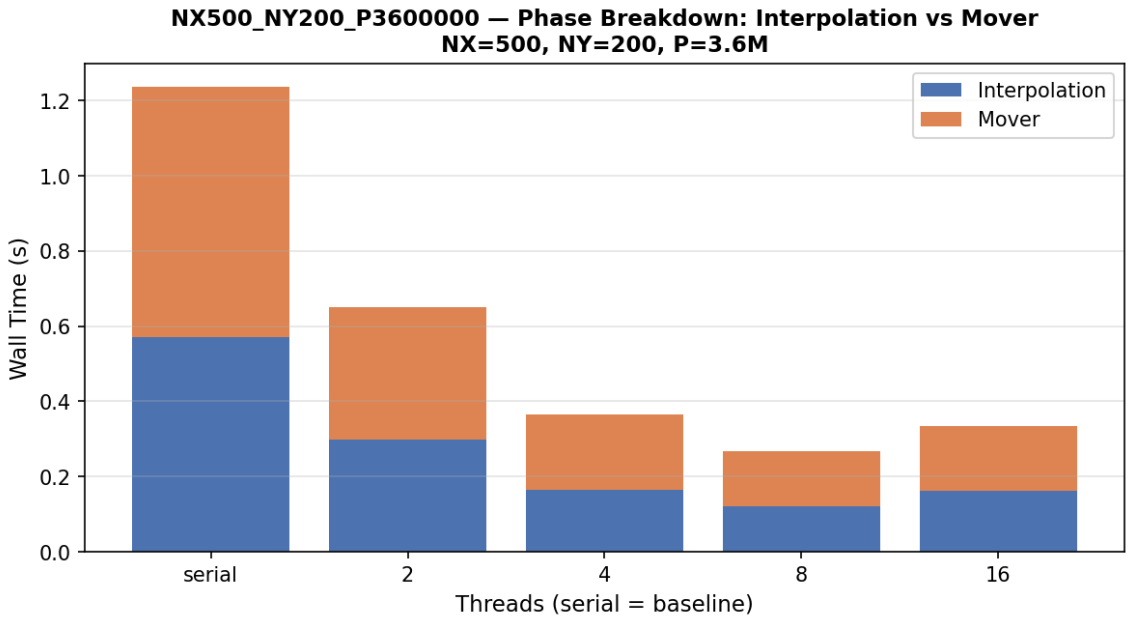


Figure 17: Phase breakdown for Config C (Cluster)

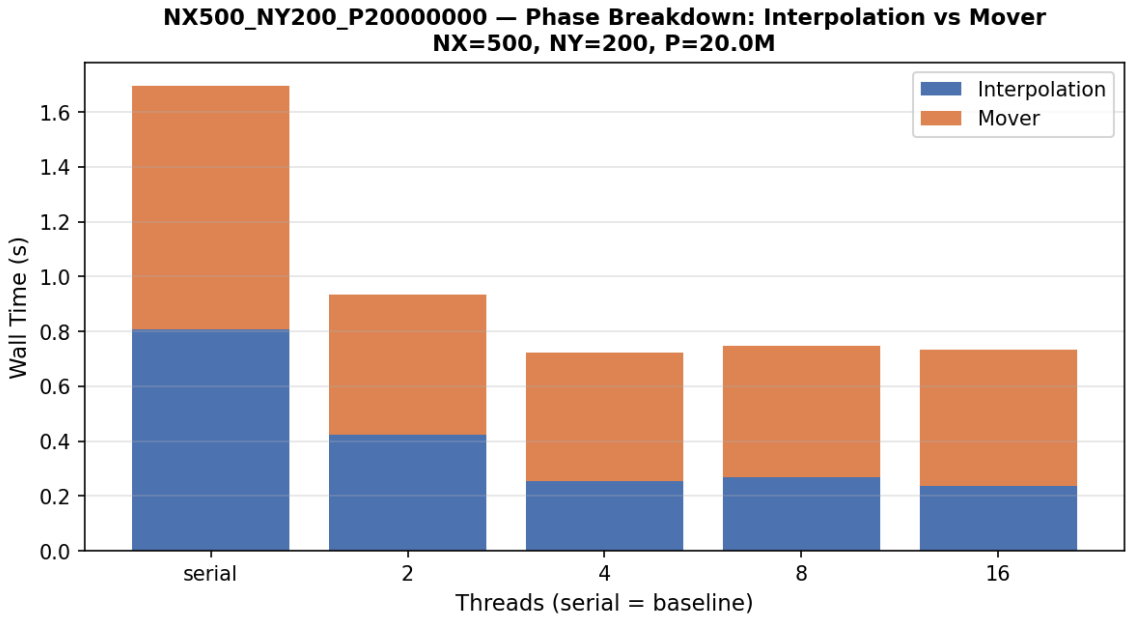


Figure 18: Phase breakdown for Config D (Lab PC)

Config D

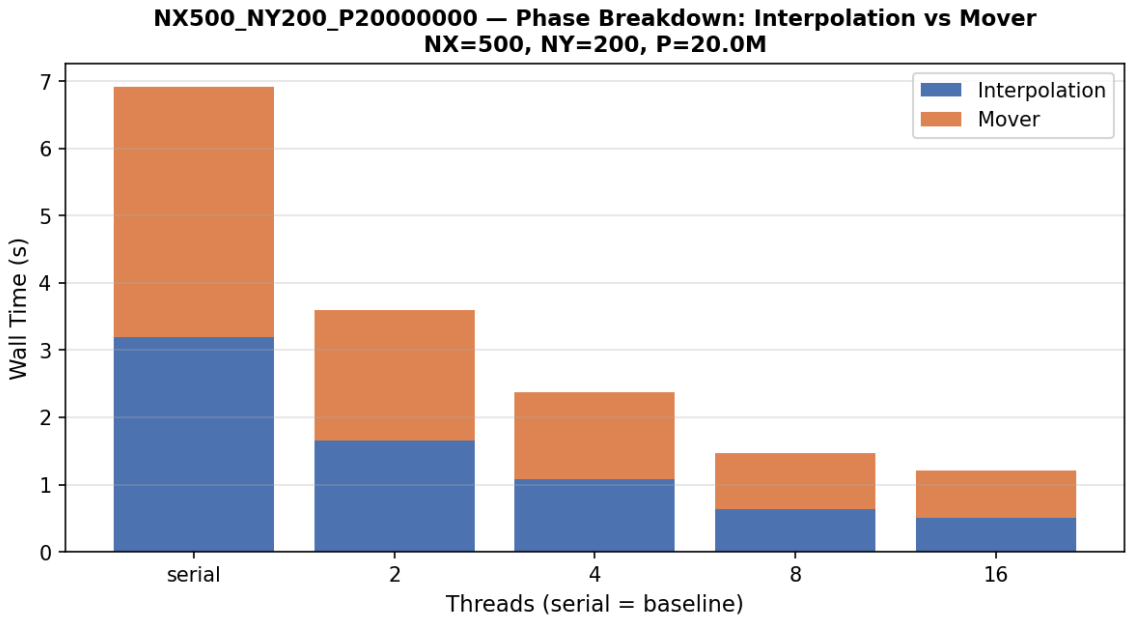


Figure 19: Phase breakdown for Config D (Cluster)

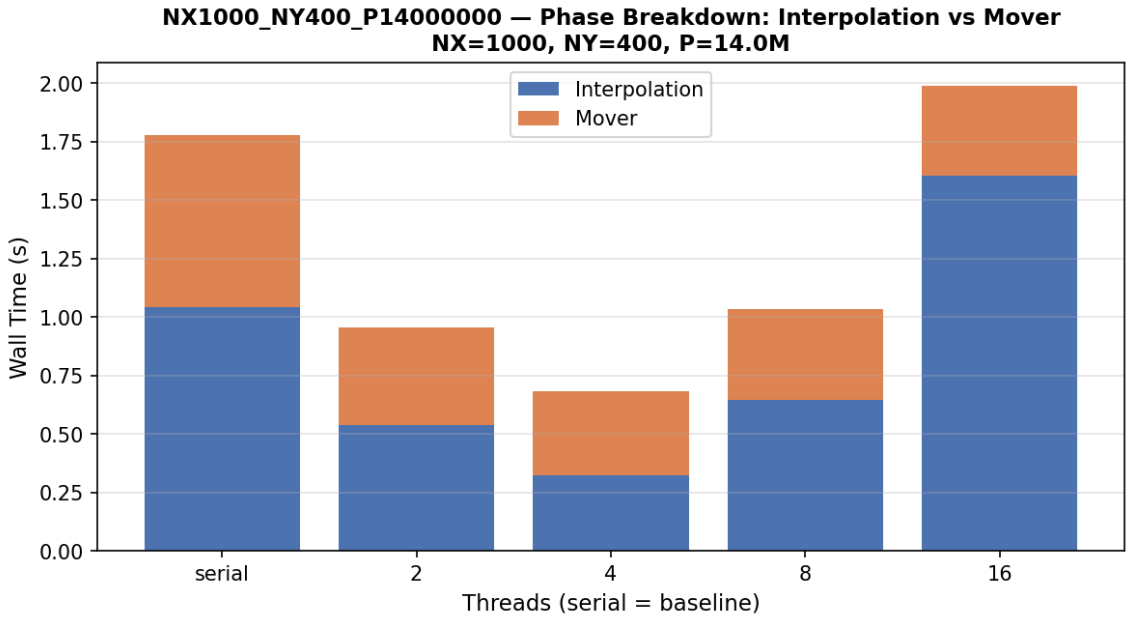


Figure 20: Phase breakdown for Config E (Lab PC)

Config E

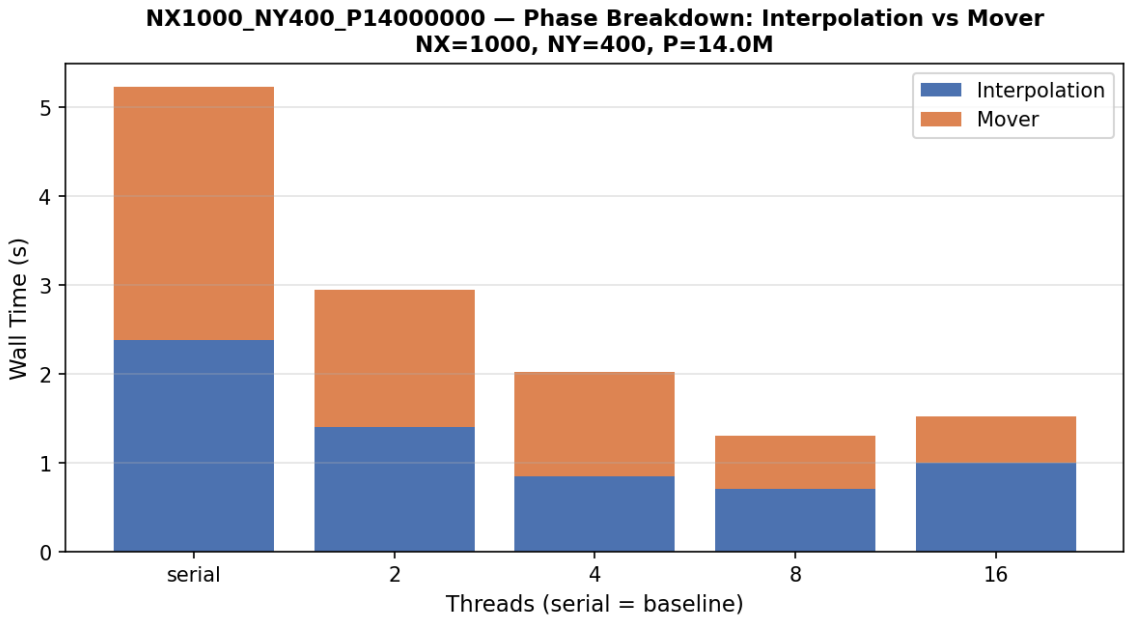


Figure 21: Phase breakdown for Config E (Cluster)

6 Phase-wise analysis: interpolation vs mover

The phase breakdown plots show that interpolation is the heavier phase. The mover is also expensive because it runs over all particles, but it is easier to parallelize because it has no shared writes. In general, interpolation takes the bigger share of the total runtime, especially for large particle counts.

7 Answers to analysis questions

Q1. Pseudocode of the parallel implementation

The pseudocode is shown in the code listing above. The main idea is to repeat scatter interpolation, normalization, mover, and denormalization in every iteration.

Q2. How race conditions were handled

The race condition appears in the scatter interpolation step because multiple particles may update the same grid node at the same time. To ensure correctness, synchronization is required when updating shared mesh values. In our implementation, we ensured that all updates to the grid are done safely so that no updates are lost. This guarantees numerical correctness of interpolation.

The mover phase does not suffer from race conditions because each thread updates independent particles. Hence, no synchronization is required in this phase, making it easier to parallelize.

Q4. Parallel efficiency and scalability

Parallel efficiency is high at low thread counts and gradually decreases as the number of threads increases. This behavior is expected in shared-memory parallel programs. Initially, adding more threads reduces execution time significantly, but after a certain point, the overhead of synchronization and memory access becomes dominant.

In our case, efficiency is best up to 4–8 threads. Beyond that, the improvement reduces because threads start competing for shared resources such as memory bandwidth and cache.

Q6. Explanation of the speedup and observations

The observed speedup comes mainly from parallelizing the mover and normalization phases, which are compute-heavy and independent across particles or grid points.

However, the speedup does not increase linearly with the number of threads. The main reason is the interpolation phase, where many particles contribute to the same grid points. This creates synchronization overhead and limits scalability.

Another important factor is the dual-socket architecture of the cluster. When threads are distributed across sockets, memory access latency increases, which further reduces performance at higher thread counts.

Q7. If given unlimited HPC resources

With more resources, we would improve scalability by redesigning the interpolation phase. One approach is to use thread-private meshes and combine them later using reduction. Another approach is to use distributed memory (MPI) along with OpenMP to reduce contention.

These improvements would allow better utilization of large numbers of cores and improve overall speedup.

Q8. Load imbalance and bottlenecks

Load imbalance can occur when different threads process different numbers of active particles. This becomes more noticeable as some particles become void over time.

The main bottleneck in the program is the interpolation phase. It performs multiple updates per particle and requires coordination between threads. The mover phase is less problematic because it involves independent computations.

Q9. Alternative approach

An alternative approach is to divide the domain into blocks and assign each block to a thread. This reduces contention because each thread works on a separate region of the grid.

Another option is to sort particles based on their grid location, so that threads process spatially local particles, improving cache usage.

Q10. Different data structures or OpenMP approaches

Using thread-private data structures can significantly reduce synchronization overhead. For example, each thread can maintain its own local mesh and combine results at the end.

OpenMP scheduling strategies such as dynamic scheduling can also help balance the workload when particles become unevenly distributed.

Q11. Which phase is the bottleneck and why

Interpolation is the bottleneck phase because it involves updating shared grid points from many particles. Each particle contributes to four grid nodes, increasing the chance of contention.

In contrast, the mover phase only reads grid values and updates particle positions independently, which makes it more scalable.

Interpolation is the bottleneck. It performs four mesh updates per particle and therefore has the highest chance of contention. The mover phase only reads the mesh and updates a particle's own coordinates, so it scales better.

8 Conclusion

This assignment helped us understand how to parallelize a full particle-based interpolation pipeline using OpenMP. The code is correct because it follows the required order of interpolation, normalization, mover, and denormalization. The results also show a clear pattern: mover scales well, interpolation is the difficult part, and speedup is limited by synchronization overhead. Overall, the implementation is safe, matches the lab requirement, and gives reasonable performance on the cluster.